

Practice 1: IEEE 802.15.4: Task scheduler, timer, data transmission

Redes para Sistemas Embebidos



ITESO

**Universidad Jesuita
de Guadalajara**

Team 7:

Sofia Davis Rodríguez

Cuauhtemoc Alejandro Herrera Pérez

Plinio Yadin Morales Rodriguez

Professor: Estephania Martínez García

Data: 23 de Septiembre, 2026

Content

Practice 1: IEEE 802.15.4: Task scheduler, timer, data transmission.....	1
Introduction	3
Repository link:	3
Theoretical Framework	4
IEEE 802.15.4	4
Network topology: PAN Coordinator vs. End Device	4
PAN ID and channel selection.....	4
Addressing modes	4
Association procedure	4
MLME and MCPS Service Access Points	5
Development	6
Modifications made to the coordinator	6
Modifications made to the end device.....	11
Conclusions	15

Introduction

Team 7 — Channel 11, PAN ID 0x7777

For this practice we had to get two FRDM-KW41Z boards talking to each other over IEEE 802.15.4 in a non-beacon-enabled star network: one board acting as the PAN Coordinator and the other as an End Device. On top of just getting them to associate, we also had to build a small "Fase 1" feature on top of the base NXP SDK demo: a counter that increases every 4 seconds, gets shown on the End Device's RGB LED with a different color per value, gets sent wirelessly to the Coordinator so it can mirror the same color on its own LED, and can also be overridden manually by pressing a switch on the End Device.

Since our team number is 7, we had to use channel 11 and PAN ID 0x7777 and make sure every device on the network had its own unique 64-bit extended MAC address (no two boards can share one, otherwise the MAC layer gets confused about who's who).

We started from the NXP MCUXpresso SDK wireless examples (`wireless_examples_ieee_802_15_4_mwa_end_device_freertos` and the equivalent coordinator example), which already implement the basic scan/associate/poll state machine, and we modified them to fit the practice's requirements. This report covers our whole process: the theory behind what we touched, what we changed in the code, and what each of us personally took away from the practice.



Repository link:

https://github.com/Cuau-herrera/P1_Redes

Theoretical Framework

IEEE 802.15.4

IEEE 802.15.4 is the standard that defines the PHY and MAC layers for low-rate, low-power wireless personal area networks (LR-WPANs). It's the foundation that things like Zigbee and Thread are built on top of, but in this practice we work directly at the raw MAC/PHY level, without any higher-layer network protocol on top.

PHY layer: handles the actual radio transmission — modulation, channel selection, and framing at the bit level. For the 2.4 GHz band (which is what the KW41Z radio uses), there are 16 available channels, numbered 11 to 26.

MAC layer: handles channel access (CSMA-CA), addressing, association/disassociation, acknowledgments, and frame formatting. This is the layer our application code talks to directly through the SAP (Service Access Point) interfaces.

Network topology: PAN Coordinator vs. End Device

IEEE 802.15.4 supports star and peer-to-peer topologies. We used a star topology, where:

The PAN Coordinator is a Full Function Device (FFD) that starts the PAN, picks the channel and PAN ID, and accepts association requests from other devices.

The End Device is a Reduced Function Device (RFD) that scans for a coordinator, associates with it, and then only exchanges data with the coordinator (never directly with other end devices).

This distinction matters because the two roles run very different state machines and firmware — that's why NXP ships them as two separate SDK example projects.

PAN ID and channel selection

Every 802.15.4 network is identified by a 16-bit PAN ID, and operates on a single fixed channel. Devices on the same PAN must all agree on both values or they'll never see each other's frames. For Team 7 we used PAN ID 0x7777 and channel 11 (following the pattern PAN ID = 0xNNNN where N is the team number).

Addressing modes

802.15.4 devices can be addressed two ways:

Extended address (64-bit): basically like a MAC address — globally unique, burned into the device or configured in firmware. Every device on the network (coordinator + all end devices) must have a different extended address, otherwise the MAC layer can't tell devices apart.

Short address (16-bit): assigned dynamically by the coordinator during association, used afterward to save bandwidth (no need to send 8 bytes of address on every single packet).

Association procedure

Before an End Device can send/receive data, it has to go through this handshake with the coordinator:

Active Scan (MLME-SCAN.request with scanType = gScanModeActive_c): the End Device broadcasts a beacon request on the target channel(s) and listens for responses from coordinators that are accepting associations.

Associate Request (MLME-ASSOCIATE.request): once a suitable coordinator is found, the End Device sends this, including a Capability Information field describing what kind of device it is (FFD/RFD, receiver-on-when-idle, whether it wants an address assigned, etc.).

Associate Confirm: the coordinator replies with either a short address (if requested) or a rejection.

MLME and MCPS Service Access Points

The application layer never touches the radio directly — everything goes through two message-passing interfaces:

MLME (MAC Layer Management Entity) SAP: handles management-type primitives — scan, associate, poll, PIB (PAN Information Base) get/set.

MCPS (MAC Common Part Sublayer) SAP: handles actual data transfer — MCPS-DATA.request to send a frame, MCPS-DATA.confirm/MCPS-DATA.indication to know it went out or arrived.

Both are asynchronous: you send a message into a queue, and the response/indication comes back later as an event, which is how the whole application is built as an event-driven state machine instead of blocking calls.

Development

Modifications made to the coordinator

```
// Equipo 7, nodo Coordinador
#define mMacExtendedAddress_c    (0x00000000000070001ULL)

/* Set the Coordinator short address */
#define mDefaultValueOfShortAddress_c    0x0000

/* Set the Coordinator PanID */
#define mDefaultValueOfPanId_c    0x7777
```

We defined `mCoordinatorMaxNodes_c` (5) and a new `tCoordinatorNodeInfo_t` struct to keep track of each associated device's short address, extended address, rxOnWhenIdle flag, and device type, laying the groundwork for supporting multiple End Devices.

```
/* Maximum number of devices that can be registered by the Coordinator. */
#define mCoordinatorMaxNodes_c 5
typedef struct
{
    uint16_t shortAddress;
    uint64_t extendedAddress;
    uint8_t rxOnWhenIdle;
    uint8_t deviceType;
    uint8_t inUse;
} tCoordinatorNodeInfo_t;
```

We initialized the Coordinator's fixed short address and PAN ID, hard-coded the logical channel to 11, and added the actual node table array (`mCoordinatorNodeTable`) plus a counter (`mCoordinatorNextShortAddress`) used to hand out unique short addresses to newly associated devices.

```
/* The short address and PAN ID of the coordinator*/
static const uint16_t mShortAddress = 0x0000;
static const uint16_t mPanId = 0x7777;
/* The current logical channel (frequency band) */
static uint8_t mLogicalChannel = 11;
/* These byte arrays stores an associated
   devices long and short addresses. */
static uint16_t mDeviceShortAddress = 0xFFFF;
static uint64_t mDeviceLongAddress = 0xFFFFFFFFFFFFFFFF;
/* Table used to retain association information for up to five devices. */
static tCoordinatorNodeInfo_t mCoordinatorNodeTable[mCoordinatorMaxNodes_c];
/* Next short address to assign to a new device. */
static uint16_t mCoordinatorNextShortAddress = 0x0001;
```

In `App_init()` we added code to clear the node table and reset the next-short-address counter, making sure the Coordinator starts every run with a clean association table.

```
/* Reset the registered-device table. */
FLib_MemSet(mCoordinatorNodeTable, 0, sizeof(mCoordinatorNodeTable));
mCoordinatorNextShortAddress = 0x0001;
/* Prepare input channels */
```

We forced the energy-detection scan to only check channel 11 instead of scanning the full default channel range, since our team was assigned a single fixed channel.

```
433 /* ChannelsToScan */
434 #ifdef gPHY_802_15_4g_d
435     pScanReq->channelPage = gChannelPageId9_c;
436     pScanReq->scanChannels[0] = (1 << 11);
437     FLib_MemSet(pScanReq->scanChannels+1, 0, sizeof(channelMask_t)-sizeof(uint32_t));
438 #else
439     pScanReq->scanChannels = (1 << 11);
440 #endif
```

This is the new `Coordinator_FindByExtended()` helper, which searches the node table for an already-registered device by its extended address, used to detect devices re-associating.

```
597 static int8_t Coordinator_FindByExtended(uint64_t extendedAddress)
598 {
599     uint8_t index;
600     for(index = 0; index < mCoordinatorMaxNodes_c; index++)
601     {
602         if(mCoordinatorNodeTable[index].inUse &&
603            (mCoordinatorNodeTable[index].extendedAddress == extendedAddress))
604         {
605             return (int8_t)index;
606         }
607     }
608     return -1;
609 }
```

This is the new `Coordinator_RegisterNode()` function, which either reuses the short address of an already-known device or assigns the next available unique short address to a newly associating device, and stores its capability info (RX-on-when-idle, device type).

```

610 static uint8_t Coordinator_RegisterNode(uint64_t extendedAddress, uint8_t capabilityInfo, uint16_t *assignedShortAddress)
611 {
612     int8_t existingIndex;
613     uint8_t newIndex;
614     if(assignedShortAddress == NULL)
615     {
616         return FALSE;
617     }
618     existingIndex = Coordinator_FindByExtended(extendedAddress);
619     if(existingIndex >= 0)
620     {
621         /* Existing extended addresses keep their original short address. */
622         *assignedShortAddress = mCoordinatorNodeTable[(uint8_t)existingIndex].shortAddress;
623         mCoordinatorNodeTable[(uint8_t)existingIndex].rxOnWhenIdle =
624             (capabilityInfo & 0x08) ? TRUE : FALSE;
625         mCoordinatorNodeTable[(uint8_t)existingIndex].deviceType =
626             (capabilityInfo & 0x02) ? TRUE : FALSE;
627         return TRUE;
628     }
629     for(newIndex = 0; newIndex < mCoordinatorMaxNodes_c; newIndex++)
630     {
631         if(!mCoordinatorNodeTable[newIndex].inUse)
632         {
633             break;
634         }
635     }
636     if(newIndex >= mCoordinatorMaxNodes_c)
637     {
638         return FALSE;

```

This is the rest of `Coordinator_RegisterNode()`, handling the case where the node table is full (returns failure) and finalizing the new entry's fields before returning the assigned short address.

```

638         return FALSE;
639     }
640     if(capabilityInfo & gCapInfoAllocAddr_c)
641     {
642         mCoordinatorNodeTable[newIndex].shortAddress = mCoordinatorNextShortAddress++;
643     }
644     else
645     {
646         mCoordinatorNodeTable[newIndex].shortAddress = 0xFFFE;
647     }
648     mCoordinatorNodeTable[newIndex].extendedAddress = extendedAddress;
649     mCoordinatorNodeTable[newIndex].rxOnWhenIdle =
650         (capabilityInfo & 0x08) ? TRUE : FALSE;
651     mCoordinatorNodeTable[newIndex].deviceType =
652         (capabilityInfo & 0x02) ? TRUE : FALSE;
653     mCoordinatorNodeTable[newIndex].inUse = TRUE;
654     *assignedShortAddress = mCoordinatorNodeTable[newIndex].shortAddress;
655     return TRUE;
656 }

```

This is the new `Coordinator_PrintNode()` function, which prints a formatted line over UART showing an associated node's index, short address, extended address, RX-on-when-idle flag, and whether it's an FFD or RFD device — used for debugging/logging each association.

```

657 static void Coordinator_PrintNode(uint8_t nodeIndex)
658 {
659     Serial_Print(interfaceId, "\r\n[Association] Node ", gAllowToBlock_d);
660     Serial_PrintHex(interfaceId, &nodeIndex, 1, gPrtHexNoFormat_c);
661     Serial_Print(interfaceId, " | Short: 0x", gAllowToBlock_d);
662     Serial_PrintHex(interfaceId,
663                     (uint8_t*)&mCoordinatorNodeTable[nodeIndex].shortAddress,
664                     2, gPrtHexNoFormat_c);
665     Serial_Print(interfaceId, " | Extended: 0x", gAllowToBlock_d);
666     Serial_PrintHex(interfaceId,
667                     (uint8_t*)&mCoordinatorNodeTable[nodeIndex].extendedAddress,
668                     8, gPrtHexNoFormat_c);
669     Serial_Print(interfaceId, " | RxOnWhenIdle: ", gAllowToBlock_d);
670     Serial_PrintHex(interfaceId, &mCoordinatorNodeTable[nodeIndex].rxOnWhenIdle, 1, gPrtHexNoFormat_c);
671     Serial_Print(interfaceId, " | DeviceType: ", gAllowToBlock_d);
672     if(mCoordinatorNodeTable[nodeIndex].deviceType)
673     {
674         Serial_Print(interfaceId, "FFD", gAllowToBlock_d);
675     }
676     else
677     {
678         Serial_Print(interfaceId, "RFD", gAllowToBlock_d);
679     }
680     Serial_Print(interfaceId, "\r\n", gAllowToBlock_d);
681 }

```

We replaced the old hard-coded short-address assignment with a call to `Coordinator_RegisterNode()`, so each associating device gets a unique short address (or is rejected with `gPanAtCapacity_c` if the table is full) instead of every device always getting the same fixed address.

```

718     FLib_MemCpy(&incomingExtendedAddress, &pMsgIn->msgData.associateInd.deviceAddress, 8);
719     incomingCapabilityInfo = pMsgIn->msgData.associateInd.capabilityInfo;
720     if(Coordinator_RegisterNode(incomingExtendedAddress, incomingCapabilityInfo, &assignedShortAddress))
721     {
722         pAssocRes->assocShortAddress = assignedShortAddress;
723         pAssocRes->status = gSuccess_c;
724     }
725     else
726     {
727         pAssocRes->assocShortAddress = 0xFFFF;
728         pAssocRes->status = gPanAtCapacity_c;
729     }

```

Inside the MCPS data-indication handler we added a detailed UART printout of each received packet (source address, length, LQI, payload), and then added logic that parses the counter digit from our custom "Counter: x" payload and mirrors it on the Coordinator's own RGB LED using the same green/red/blue/magenta color mapping as the End Device, giving a visual confirmation on the Coordinator side of what value each End Device is broadcasting.

```

813  /* Imprimir detalles de la telemetria para el reporte */
814  Serial_Print(interfaceId, "\r\n[RX Data] From: 0x", gAllowToBlock_d);
815  Serial_PrintHex(interfaceId, (uint8_t*)&pMsgIn->msgData.dataInd.srcAddr, 2, gPrtHexNoFormat_c);
816  Serial_Print(interfaceId, " | Length: ", gAllowToBlock_d);
817  Serial_PrintHex(interfaceId, (uint8_t*)&pMsgIn->msgData.dataInd.msduLength, 1, gPrtHexNoFormat_c);
818  Serial_Print(interfaceId, " | LQI: ", gAllowToBlock_d);
819  Serial_PrintHex(interfaceId, (uint8_t*)&pMsgIn->msgData.dataInd.mpduLinkQuality, 1, gPrtHexNoFormat_c);
820  Serial_Print(interfaceId, " | Payload: ", gAllowToBlock_d);
821  /* Imprimir el contenido del paquete */
822  Serial_SyncWrite( interfaceId, pMsgIn->msgData.dataInd.pMsdu, pMsgIn->msgData.dataInd.msduLength );
823  Serial_Print(interfaceId, "\r\n", gAllowToBlock_d);
824  /* Cambiar el LED RGB del Coordinador segun el contador recibido */
825  if(pMsgIn->msgData.dataInd.msduLength >= 10)
826  {
827      uint8_t counterVal = pMsgIn->msgData.dataInd.pMsdu[9] - '0';
828      TurnOffLeds();
829      /* board.h maps the RGB LED as LED2=Red, LED3=Green, LED4=Blue */
830      switch(counterVal % 4)
831      {
832          case 0: LED_TurnOnLed(LED3); break; /* Verde */
833          case 1: LED_TurnOnLed(LED2); break; /* Rojo */
834          case 2: LED_TurnOnLed(LED4); break; /* Azul */
835          case 3:                               /* Magenta */
836              LED_TurnOnLed(LED2);
837              LED_TurnOnLed(LED4);
838              break;
839      }
840  }

```

Modifications made to the end device

We hard-coded a unique 64-bit extended MAC address for the End Device (0x00000000000070002ULL), replacing the default broadcast-style value so our node has an identity distinct from the Coordinator's address on the Team 7 network.

```
28 /* Equipo 7, nodo End Device */
29 #define mMacExtendedAddress_c (0x00000000000070002ULL)
```

This is the PHY branch selector for the channel mask (gPHY_802_15_4g_d); we kept the original 17-bit mask here since our board doesn't use the 802.15.4g PHY, and instead restricted the actual scan mask used by our setup in the #else branch right below to a single fixed channel (11), so the End Device only scans and joins on the channel our team was assigned.

```
39 #ifdef gPHY_802_15_4g_d
40 #define mDefaultValueOfChannel_c (0x0001FFFF)
```

We added the state needed for our periodic counter feature: a dedicated timer ID (mCounterTimerId), the current counter value (mCounterValue, cycling 0-3), a variable to hold a manual override value coming from a switch press (mSwitchOverrideValue), and a 4000 ms poll interval constant used to drive the counter.

```
146 /* Fase 1: timer de 4s, valor actual del contador [0-3], y override por switch */
147 static tmrTimerID_t mCounterTimerId = gTmrInvalidTimerID_c;
148 static uint8_t mCounterValue = 0;
149 static uint8_t mSwitchOverrideValue;
150 #define mCounterPollInterval_c 4000
```

In the NVM-restore code path (when the device reconnects using saved association data instead of doing a fresh scan/associate), we start the same 4-second counter timer, reset the counter to 0, and immediately update the RGB LED, so the counter feature works correctly even after a restored session.

```
457 /* Fase 1: iniciar el timer de 4s del contador tambien en este camino (restore NVM) */
458 mCounterTimerId = TMR_AllocateTimer();
459 mCounterValue = 0;
460 App_UpdateCounterLed(mCounterValue);
461 TMR_StartIntervalTimer(mCounterTimerId, mCounterPollInterval_c, App_CounterTimerCallback, NULL);
462 gState = stateListen;
463 }
464 else
465 {
```

Right after a successful association with the Coordinator (the normal first-time join path), we allocate and start the 4-second counter timer and light the LED for counter value 0, kicking off the periodic counter behavior as soon as the device joins the network.

```
569 /* Fase 1: iniciar el timer de 4s del contador */
570 mCounterTimerId = TMR_AllocateTimer();
571 mCounterValue = 0;
572 App_UpdateCounterLed(mCounterValue);
573 TMR_StartIntervalTimer(mCounterTimerId, mCounterPollInterval_c, App_CounterTimerCallback, NULL);
```

Inside the main "listen" state's event loop we added two new event handlers: one that fires every 4 seconds to advance the counter (mod 4), update the LED color, and send a status packet to the Coordinator; and another that reacts to a switch press by immediately overriding the counter to a fixed value, updating the LED, sending the packet, and restarting the 4-second timer so the cycle continues from the new value

```

613      /* Fase 1: contador periodico de 4s */
614      if (ev & gAppEvtCounterTimerExpired_c)
615      {
616          mCounterValue = (mCounterValue + 1) % 4;
617          App_UpdateCounterLed(mCounterValue);
618          App_SendCounterPacket(mCounterValue);
619      }
620      /* Fase 1: override inmediato por switch, reinicia el timer de 4s */
621      if (ev & gAppEvtSwitchOverride_c)
622      {
623          mCounterValue = mSwitchOverrideValue;
624          App_UpdateCounterLed(mCounterValue);
625          App_SendCounterPacket(mCounterValue);
626          TMR_StartIntervalTimer(mCounterTimerId, mCounterPollInterval_c, App_CounterTimerCallback, NULL);
627      }

```

This is the new App_UpdateCounterLed() function, which maps the counter value (0-3) to a specific RGB LED color — green, red, blue, or magenta — using the board's discrete LED pins (LED2=Red, LED3=Green, LED4=Blue).

```

1201 * Fase 1: reflects mCounterValue on the RGB LED (0=Green,1=Red,2=Blue,3=Magenta)
1202 *****/
1203 static void App_UpdateCounterLed(uint8_t counter)
1204 {
1205     TurnOffLeds();
1206     /* board.h mapea el LED RGB como LED2=Rojo, LED3=Verde, LED4=Azul */
1207     switch(counter % 4)
1208     {
1209         case 0: LED_TurnOnLed(LED3); break;          /* Verde */
1210         case 1: LED_TurnOnLed(LED2); break;          /* Rojo */
1211         case 2: LED_TurnOnLed(LED4); break;          /* Azul */
1212         case 3:                                     /* Magenta */
1213             LED_TurnOnLed(LED2);
1214             LED_TurnOnLed(LED4);
1215             break;
1216     }
1217 }

```

This is the new App_CounterTimerCallback(), the timer interrupt callback that simply signals the application task with the counter-expired event so the actual counter logic runs safely in task context.

```

1220 * Fase 1: fires every 4 seconds to advance the counter
1221 *****/
1222 static void App_CounterTimerCallback(void *param)
1223 {
1224     (void)param;
1225     OSA_EventSet(mAppEvent, gAppEvtCounterTimerExpired_c);
1226 }

```

This is the beginning of the new `App_SendCounterPacket()` function, which builds a small "Counter: x" payload and sends it as an MCPS-Data request addressed to the Coordinator, allocating a message buffer and filling in the source/destination addressing fields.

```

1229 * Page 1: builds and sends the "Counter: x" MCPS-Data request to the coordinator
1230 *****/
1231 static void App_SendCounterPacket(uint8_t counter)
1232 {
1233     nwkToMcpMessage_t *pCounterPacket;
1234     uint8_t payload[10] = "Counter: 0";
1235     payload[9] = (uint8_t)('0' + (counter % 4));
1236
1237     if(mcPendingPackets < mDefaultValueOfMaxPendingDataPackets_c)
1238     {
1239         pCounterPacket = MSG_Alloc(sizeof(nwkToMcpMessage_t) + sizeof(payload));
1240         if(pCounterPacket != NULL)
1241         {
1242             pCounterPacket->msgType = gMcpDataReq_c;
1243             pCounterPacket->msgData.dataReq.pMsd = (uint8_t*)(pCounterPacket->msgData.dataReq.pMsd) +
1244                 sizeof(pCounterPacket->msgData.dataReq.pMsd);
1245             FLib_MemCpy(pCounterPacket->msgData.dataReq.pMsd, payload, sizeof(payload));
1246             FLib_MemCpy(&pCounterPacket->msgData.dataReq.dstAddr, &mCoordInfo.coordAddress, 8);
1247             FLib_MemCpy(&pCounterPacket->msgData.dataReq.srcAddr, &mMyAddress, 8);
1248             FLib_MemCpy(&pCounterPacket->msgData.dataReq.dstPanId, &mCoordInfo.coordPanId, 2);
1249             FLib_MemCpy(&pCounterPacket->msgData.dataReq.srcPanId, &mCoordInfo.coordPanId, 2);
1250             pCounterPacket->msgData.dataReq.dstAddrMode = mCoordInfo.coordAddrMode;
1251             pCounterPacket->msgData.dataReq.srcAddrMode = mAddrMode;
1252             pCounterPacket->msgData.dataReq.msdLength = sizeof(payload);
1253             pCounterPacket->msgData.dataReq.txOptions = gMacTxOptionsAck_c;
1254             pCounterPacket->msgData.dataReq.msdHandle = mMsdHandle++;

```

This is the rest of App_SendCounterPacket(), where the packet is queued for transmission through the MAC layer and the same "Counter: x" text is printed over UART for debugging/logging purposes.

```

1254         pCounterPacket->msgData.dataReq.msduHandle = mMsdHandle++;
1255         pCounterPacket->msgData.dataReq.securityLevel = gMacSecurityNone_c;
1256         (void)NWK_MCPS_SapHandler(pCounterPacket, macInstance);
1257         mcPendingPackets++;
1258     }
1259 }
1260 Serial_Print(interfaceId, "\r\nSent: ", gAllowToBlock_d);
1261 Serial_SyncWrite(interfaceId, payload, sizeof(payload));
1262 Serial_Print(interfaceId, "\r\n", gAllowToBlock_d);
1263 }

```

These are the new SW1/SW2 key-press handlers: when the device is already in the listening state, pressing SW1 forces the counter to 0 and pressing SW2 forces it to 2, each triggering the switch-override event; if the device is still in the init state, the same key presses fall back to their original behavior of waking the device from the flashing-LED startup state.

```
1279     Verificar en la tarjeta cual botón corresponde a "SW3"/"SW4" del enunciado. */
1280     case gKBD_EventSW1_c:
1281         if(gState == stateListen)
1282         {
1283             mSwitchOverrideValue = 0; /* SW3 -> Counter = 0 */
1284             OSA_EventSet(mAppEvent, gAppEvtSwitchOverride_c);
1285             break;
1286         }
1287         if(gState == stateInit)
1288         {
1289             LED_StopFlashingAllLeds();
1290             OSA_EventSet(mAppEvent, gAppEvtDummyEvent_c);
1291         }
1292         break;
1293     case gKBD_EventSW2_c:
1294         if(gState == stateListen)
1295         {
1296             mSwitchOverrideValue = 2; /* SW4 -> Counter = 2 */
1297             OSA_EventSet(mAppEvent, gAppEvtSwitchOverride_c);
1298             break;
1299         }
1300         if(gState == stateInit)
1301         {
1302             LED_StopFlashingAllLeds();
1303             OSA_EventSet(mAppEvent, gAppEvtDummyEvent_c);
1304         }
```

Conclusions

Sofia Davis: For me, this practice was a good way to understand how an embedded device can communicate using the 802.15.4 MAC layer. I learned more about how the End Device associates with the coordinator and how the counter can be sent as data. I also liked seeing how the value of the counter was connected to the RGB LED, because it made it easier to know what was happening without only looking at the terminal. There were some small problems while changing the original code but testing the program and checking the messages helped us find what was happening.

Cuauhtemoc Herrera: This practice helped me understand better how communication works in an IEEE 802.15.4 network. I was able to see how the End Device connects to the coordinator and how the messages are sent through the network. Working with the timer, switches, and LEDs also helped me understand how different events can work together in the same application. Some parts of the code were a little confusing, especially when modifying the original project, but after testing it several times, it became easier to understand how everything was connected.

Plinio Yadin: This practice helped me get a better idea of how the different parts of an embedded wireless application work together. The timer, switches, LEDs, and MAC messages all had a specific purpose, and seeing them work together made the code easier to understand. I also learned that small changes in the original project can affect the way the device behaves, so testing each modification was important. Seeing messages like “Counter: 1”, “Counter: 2”, and so on in the terminal was also useful because we could verify that the device was actually sending the information.